

Yenepoya Institute of Arts , Science , Commerce & Management

Project – High Level Design

on

Natural Language Interface for Health Infrastructure

Student Name: VISHNU K SANTHOSH

Industry Mentor: Mr.Sumit Kumar

Guided by: Mrs.Deeksha Shetty

Table of Contents

- 1 Introduction
 - 1.1 Scope of the Document
 - 1.2 Intended Audience
 - 1.3 System Overview
- 2 System Design
 - 2.1 Application Design
 - 2.2 Process Flow
 - 2.3 Information Flow
 - 2.4 Components Design
 - 2.5 Key Design Considerations
 - 2.6 API Catalogue
- 3 Data Design
 - 3.1 Data Model
 - 3.2 Data Access Mechanism
 - 3.3 Data Retention Policies
 - 3.4 Data Migration
- 4 Interfaces
- 5 State and Session Management
- 6 Caching
- 7 Non-Functional Requirements
 - 7.1 Security Aspects
 - 7.2 Performance Aspects
- 8 References

1. Introduction

1.1 Scope of the Document

This document presents the High-Level Design (HLD) for the **Natural Language Interface (NLI) for Healthcare Infrastructure** system. It describes the overall architecture, major system components, data flow, integrations, and operational considerations.

The scope includes:

- Functional overview of system modules
- Interaction between components
- Data handling and storage strategies
- Integration with external healthcare systems
- Security and performance requirements

This document acts as a reference for system design, development, deployment, and maintenance.

1.2 Intended Audience

This document is intended for multiple stakeholders:

- **Software Engineers** – to understand system architecture and module responsibilities
 - **System Architects** – for validating design decisions and scalability
 - **QA/Test Engineers** – for designing test cases based on workflows
 - **Project Managers** – for planning and tracking development phases
 - **Healthcare IT Staff** – for deployment and operational understanding
-

1.3 System Overview

The system is an AI-driven healthcare assistant that enables users to interact with healthcare services using **natural language (text or voice)**.

Core Capabilities:

- Intelligent chatbot for healthcare queries
- Appointment scheduling and management
- Medical record access and retrieval
- Doctor recommendation based on symptoms
- Automated FAQs and guidance

Technology Stack Overview:

- **Frontend:** Web / Mobile UI (React, Flutter, etc.)
- **Backend:** RESTful services (Node.js / Python Flask / Django)
- **AI Layer:** NLP models (spaCy, Transformers, BERT-like models)
- **Database:** SQL + NoSQL hybrid
- **Integration:** APIs with hospital systems and EHR

2. System Design

2.1 Application Design

The system follows a **layered microservices-based architecture**:

1. Presentation Layer

- Chat-based UI
- Voice assistant interface
- Accessible UI for elderly users

2. Application Layer (Service Layer)

- NLP Processing Service
- Appointment Management Service
- Patient Data Service
- Notification Service
- Authentication Service

3. Data Layer

- Relational Database (structured healthcare data)
 - NoSQL Database (chat logs, NLP data)
 - External APIs (EHR systems)
-

2.2 Process Flow

Detailed Workflow:

1. User submits query (text/voice)
2. Voice → Speech-to-text conversion (if needed)
3. Query preprocessing (cleaning, tokenizing)
4. Intent classification using ML model
5. Entity extraction (NER – Named Entity Recognition)
6. Context handling (previous conversation memory)
7. Mapping to backend service
8. Database/API interaction
9. Business logic execution
10. Response generation
11. Output delivered (text/voice)

2.3 Information Flow

Logical Data Movement:

- User Input → NLP Engine
- NLP Engine → Intent & Entity Layer
- Intent Layer → Service Dispatcher
- Service Layer ↔ Database / APIs
- Processed Output → Response Engine → UI

Bidirectional Flow:

- User ↔ System (interactive dialogue loop)
 - System ↔ External APIs (real-time updates)
-

2.4 Components Design

1. User Interface Module

- Chat UI
 - Voice command UI
 - Dashboard for appointments
-

2. Speech Recognition Module

- Converts speech → text
 - Uses APIs like Google Speech or open-source engines
-

3. NLP Processing Engine

- Tokenization
 - POS tagging
 - Dependency parsing
-

2.5 Key Design Considerations

Scalability

- Microservices allow independent scaling
- Cloud deployment (AWS/Azure/GCP)

Accuracy

- Continuous training of NLP models
- Feedback-based learning

Availability

- Load balancing
- Failover systems

Maintainability

- Modular architecture
- Clear API boundaries

Interoperability

- Supports HL7/FHIR healthcare standards

User Accessibility

- Multi-language support
- Voice interaction

2.6 API Catalogue

1. Authentication APIs

- POST /login
 - POST /register
 - POST /verify-otp
-

2. Appointment APIs

- GET /doctors
 - POST /appointments
 - PUT /appointments/{id}
 - DELETE /appointments/{id}
-

3. Medical Records APIs

- GET /records
 - GET /records/{id}
 - POST /upload-report
-

4. NLP APIs

- POST /analyze-query
 - POST /extract-entities
-

5. Notification APIs

- POST /send-email
- POST /send-sms

3. Data Design

3.1 Data Model

Entity Relationship Overview:

- Patient ↔ Appointment ↔ Doctor
 - Patient ↔ Reports
 - Doctor ↔ Specialization
-

Sample Tables:

Patients Table

- patient_id (PK)
- name
- age
- contact

Doctors Table

- doctor_id (PK)
- specialization
- availability

Appointments Table

- appointment_id
 - patient_id
 - doctor_id
 - date_time
-

3.2 Data Access Mechanism

- REST APIs
- ORM (Object Relational Mapping)
- Secure tokens for authentication
- Role-based access control

3.3 Data Retention Policies

- Patient data stored securely long-term
 - Logs stored for audit and debugging
 - Data anonymization for analytics
 - Backup policies implemented
-

3.4 Data Migration

- Legacy system integration
 - Bulk data import
 - Schema transformation
 - Data validation
-

4. Interfaces

User Interfaces

- Web UI
 - Mobile App
 - Voice Assistant
-

External Interfaces

- Hospital Information Systems
 - Electronic Health Records (EHR)
 - Payment Gateway (optional)
 - Notification Systems
-

Internal Interfaces

- Microservices communication (REST/JSON)
 - Message queues (Kafka/RabbitMQ)
-

5. State and Session Management

State Management

- Conversation state tracking
- Workflow state (appointment progress)

Session Handling

- Token-based sessions (JWT)
 - Session timeout policies
 - Secure session storage
-

6. Caching

Caching Strategies

- In-memory caching (Redis)
- Frequently accessed data:
 - Doctor lists
 - FAQs
 - User sessions

Benefits

- Reduced latency
 - Lower database load
 - Faster response times
-

7. Non-Functional Requirements

7.1 Security Aspects

- Data encryption (AES, HTTPS)

- Secure APIs
 - Authentication & authorization
 - Compliance with healthcare standards
 - Protection against attacks (SQL injection, XSS)
-

7.2 Performance Aspects

- Response time < 2–3 seconds
 - High throughput handling
 - Auto-scaling cloud infrastructure
 - Efficient NLP processing
 - Load balancing
-

8. References

- Google
- Wikipedia
- ChatGpt , Gemini etc.....